

Université de Abdelhamid Mehri Constantine 2

Faculté de NTIC

Département de IFA

Techniques de conception d'algorithmes et le problème du sac à dos

Rapport de projet

Réalisé par :

- Benaïssa Roumeïssa
- Hamdane Salsabil

Encadré par :

- Siad Malak

Année universitaire : 2025–2026

Table des matières

Introduction	2
0.1 Le problème du sac à dos	3
0.2 Description et implémentation des techniques de conception d'algorithmes pour le problème du sac à dos	4
0.2.1 Algorithmes gloutons classiques (Greedy Classique)	4
0.2.2 Algorithme Glouton Randomisé (Greedy Randomized Algorithm) .	10
0.2.3 La programmation dynamique	16
0.3 La comparaison entre les techniques de conception d'algorithme du pro- blème sac à dos	25
Conclusion	26

Introduction

Dans notre monde actuel, nous sommes confrontés à de nombreux problèmes complexes dans divers domaines tels que l'informatique, l'économie et l'industrie. Ces problèmes nécessitent des méthodes efficaces et optimales pour trouver des solutions appropriées. C'est dans ce contexte que les *techniques de conception d'algorithmes* (*Algorithm Design Techniques*) ont été développées. Ces techniques visent à concevoir des solutions qui offrent les meilleures performances possibles en termes de précision, de rapidité d'exécution et d'utilisation des ressources.

Parmi ces approches, on distingue plusieurs méthodes, chacune possédant ses propres caractéristiques et domaines d'application. Dans ce travail, nous nous intéressons principalement à deux grandes familles de techniques :

1. Les algorithmes gloutons (*Greedy Algorithms*)
2. La programmation dynamique (*Dynamic Programming*)

=

Nous avons également étudié une version améliorée de l'algorithme glouton, appelée *algorithme glouton randomisé* (*Greedy Randomized Algorithm*), qui introduit un élément d'aléatoire afin d'améliorer la qualité des solutions et d'éviter les optima locaux.

Pour appliquer concrètement ces techniques et les comparer, nous avons choisi le *problème du sac à dos* (*Knapsack Problem*) comme exemple représentatif. Ce problème est l'un des cas classiques de l'optimisation combinatoire et constitue un excellent support pour évaluer les performances de différentes méthodes de conception d'algorithmes.

L'étude de ce problème nous permettra de comprendre les différences de fonctionnement entre ces algorithmes, d'analyser leur impact sur la qualité et la rapidité des solutions.

0.1 Le problème du sac à dos

Définition

Le problème du sac à dos est un problème d'optimisation classique en informatique et en mathématiques. Il consiste à choisir, parmi un ensemble d'objets, ceux qui doivent être placés dans un sac afin de **maximiser la valeur totale** tout en respectant une **contrainte de capacité maximale**.

Description du problème

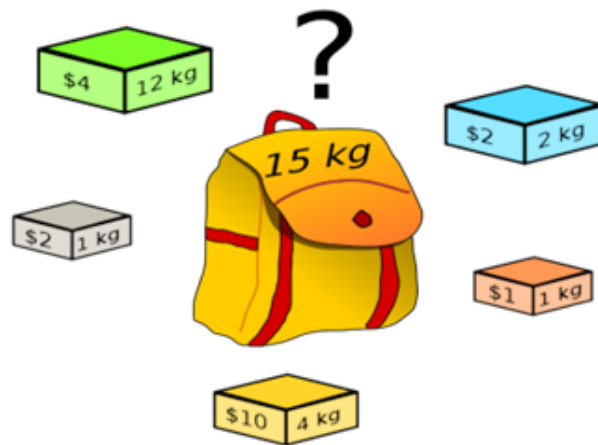
On dispose :

- d'un sac de capacité maximale C ,
- et d'une liste d'objets, chacun ayant un **poids** w_i et une **valeur** v_i .

L'objectif est de déterminer quels objets placer dans le sac afin que :

$$\sum w_i \leq C \quad \text{et que} \quad \sum v_i \text{ soit maximale.}$$

Autrement dit, il s'agit de remplir le sac le plus efficacement possible en termes de valeur, sans dépasser sa capacité.



Principales variantes

- **Problème du sac à dos 0/1 (0/1 Knapsack Problem)** : Chaque objet peut être soit inclus (1), soit non inclus (0).
- **Problème du sac à dos fractionnaire (Fractional Knapsack Problem)** : Les objets peuvent être divisés en fractions, c'est-à-dire qu'on peut prendre une partie de chaque objet.

0.2 Description et implémentation des techniques de conception d'algorithmes pour le problème du sac à dos

0.2.1 Algorithmes gloutons classiques (Greedy Classique)

Définition

Les algorithmes gloutons résolvent des problèmes en effectuant des choix **localement optimaux** à chaque étape, en espérant que ces choix mèneront à une **solution globalement optimale**. Autrement dit, à chaque étape, l'algorithme prend la meilleure décision possible sans tenir compte des conséquences à long terme.

Algorithme général d'un algorithme glouton

```
function Greedy(C : Set) : set
S ← vide
while ¬Solution(S) ET C ne pas vide do
    x ← item in C which max(Select(X))    ce s'est le choix clé
    C ← C - {x}
    if feasible(S U {x}) then
        S ← S U {x}
    if Solution(S)
        return S
    else
        return "No solution!"
```

Le critère de rapport(valeur/poid)

Implémentation en Python

L'implémentation suivante applique la méthode gloutonne classique pour le problème du sac à dos (Knapsack Problem), en choisissant les objets selon le rapport valeur/poids :

```
def Solution(limit):
    return limit == 0

def Select(items):
    items.sort(key=lambda x: x[2], reverse=True)
    return items[0]
```

```

def remove_item(x, items):
    if x in items:
        items.remove(x)
    return items

def feasible(weight, limit):
    return weight <= limit

def list_items(weights, values):
    items = []
    for i in range(len(weights)):
        weight = weights[i]
        value = values[i]
        ratio = value / weight
        items.append((weight, value, ratio))
    return items

def greedy_knapsack(items, capacity):
    selected = []
    limit = capacity

    while not Solution(limit) and items:
        weight, value, ratio = Select(items)
        items = remove_item((weight, value, ratio), items)
        if feasible(weight, limit):
            selected.append((weight, value, ratio))
            limit -= weight
    return selected if selected else []

```

Les résultats

Fichier	Nombre d'objets	Valeur totale	Poids total	Temps d'exécution (s)
knapsack _{test} 50.txt	12	474	328	0.0014820098876953125 sc
knapsack _{test} 100.txt	26	910	625	0.0022661685943603516 sc
knapsack _{test} 1000.txt	258	9256	6849	0.035416603088378906 sc

TABLE 1 – Résumé des résultats pour différents fichiers/tailles d'objets.

```
Capacity: 350
Selected items (weight, value, ratio):
Weight: 10, Value: 36, Ratio: 3.60
Weight: 11, Value: 38, Ratio: 3.45
Weight: 16, Value: 43, Ratio: 2.69
Weight: 28, Value: 50, Ratio: 1.79
Weight: 17, Value: 27, Ratio: 1.59
Weight: 26, Value: 38, Ratio: 1.46
Weight: 38, Value: 50, Ratio: 1.32
Weight: 22, Value: 26, Ratio: 1.18
Weight: 38, Value: 42, Ratio: 1.11
Weight: 44, Value: 48, Ratio: 1.09
Weight: 42, Value: 43, Ratio: 1.02
Weight: 36, Value: 33, Ratio: 0.92
Number of selected items: 12
Total value: 474
Total weight: 328
Execution time: 0.0018625259399414062 seconds
```

FIGURE 1 – Résultats pour 50 objets

```
Capacity: 637
Selected items (weight, value, ratio):
Weight: 10, Value: 48, Ratio: 4.80
Weight: 11, Value: 50, Ratio: 4.55
Weight: 12, Value: 41, Ratio: 3.42
Weight: 10, Value: 23, Ratio: 2.30
Weight: 22, Value: 47, Ratio: 2.14
Weight: 24, Value: 50, Ratio: 2.08
Weight: 15, Value: 27, Ratio: 1.80
Weight: 27, Value: 42, Ratio: 1.56
Weight: 31, Value: 45, Ratio: 1.45
Weight: 28, Value: 37, Ratio: 1.32
Weight: 35, Value: 46, Ratio: 1.31
Weight: 34, Value: 44, Ratio: 1.29
Weight: 18, Value: 23, Ratio: 1.28
Weight: 19, Value: 24, Ratio: 1.26
Weight: 25, Value: 31, Ratio: 1.24
Weight: 16, Value: 19, Ratio: 1.19
Weight: 30, Value: 45, Ratio: 1.18
Weight: 12, Value: 14, Ratio: 1.17
Weight: 32, Value: 37, Ratio: 1.16
Weight: 41, Value: 47, Ratio: 1.15
Weight: 30, Value: 34, Ratio: 1.13
Weight: 16, Value: 18, Ratio: 1.12
Weight: 43, Value: 47, Ratio: 1.09
Weight: 34, Value: 34, Ratio: 1.00
Weight: 32, Value: 30, Ratio: 0.94
Weight: 10, Value: 7, Ratio: 0.70
Number of selected items: 26
Total value: 910
Total weight: 625
Execution time: 0.0022661685943603516 seconds
```

FIGURE 2 – Résultats pour 100 objets

```
Weight: 35, Value: 34, Ratio: 0.97
Weight: 31, Value: 30, Ratio: 0.97
Weight: 51, Value: 40, Ratio: 0.78
Weight: 24, Value: 23, Ratio: 0.96
Weight: 45, Value: 43, Ratio: 0.96
Weight: 22, Value: 21, Ratio: 0.95
Weight: 35, Value: 33, Ratio: 0.94
Weight: 17, Value: 16, Ratio: 0.94
Weight: 34, Value: 32, Ratio: 0.94
Weight: 33, Value: 31, Ratio: 0.94
Weight: 32, Value: 30, Ratio: 0.94
Weight: 31, Value: 29, Ratio: 0.94
Weight: 28, Value: 26, Ratio: 0.93
Weight: 25, Value: 23, Ratio: 0.92
Weight: 55, Value: 50, Ratio: 0.91
Weight: 32, Value: 29, Ratio: 0.91
Weight: 21, Value: 19, Ratio: 0.90
Weight: 42, Value: 38, Ratio: 0.90
Weight: 31, Value: 28, Ratio: 0.90
Weight: 31, Value: 28, Ratio: 0.90
Weight: 41, Value: 37, Ratio: 0.90
Weight: 51, Value: 46, Ratio: 0.90
Weight: 30, Value: 27, Ratio: 0.90
Weight: 20, Value: 18, Ratio: 0.90
Weight: 50, Value: 45, Ratio: 0.90
Weight: 40, Value: 43, Ratio: 0.90
Weight: 40, Value: 43, Ratio: 0.90
Weight: 56, Value: 50, Ratio: 0.89
Weight: 37, Value: 33, Ratio: 0.89
Weight: 46, Value: 41, Ratio: 0.89
Weight: 45, Value: 40, Ratio: 0.89
Weight: 53, Value: 47, Ratio: 0.89
Weight: 35, Value: 31, Ratio: 0.89
Weight: 17, Value: 15, Ratio: 0.88
Weight: 25, Value: 22, Ratio: 0.88
Weight: 39, Value: 34, Ratio: 0.87
Weight: 31, Value: 27, Ratio: 0.87
Weight: 54, Value: 47, Ratio: 0.87
Weight: 23, Value: 20, Ratio: 0.87
Weight: 30, Value: 26, Ratio: 0.87
Weight: 43, Value: 37, Ratio: 0.86
Weight: 57, Value: 40, Ratio: 0.86
Weight: 47, Value: 40, Ratio: 0.85
Weight: 40, Value: 34, Ratio: 0.85
Weight: 33, Value: 28, Ratio: 0.85
Weight: 26, Value: 22, Ratio: 0.85
Weight: 30, Value: 32, Ratio: 0.84
Weight: 30, Value: 32, Ratio: 0.84
Weight: 30, Value: 32, Ratio: 0.84
Weight: 44, Value: 37, Ratio: 0.84
Weight: 37, Value: 31, Ratio: 0.84
Weight: 37, Value: 31, Ratio: 0.84
Weight: 12, Value: 10, Ratio: 0.83
Weight: 53, Value: 44, Ratio: 0.83
Number of selected items: 258
Total value: 9256
Total weight: 6049
Execution time: 0.035416683088378006 seconds
```

FIGURE 3 – Résultats pour 1000 objets

Conclusion

À partir du tableau ci-dessus, nous pouvons tirer la conclusion suivante :

- Le temps d'exécution de l'algorithme glouton augmente avec le nombre d'objets à traiter.

Étude de l'effet de la fonction de sélection (Select) sur la performance

Après avoir exécuté la version classique de l'algorithme glouton en utilisant le critère Valeur/Poids (Value/Weight Ratio), nous avons réalisé des expériences supplémentaires afin d'évaluer l'impact du changement de la fonction de sélection (Select) sur la durée d'exécution et sur la qualité de la solution finale.

Nous avons testé deux critères différents dans la même structure générale de l'algorithme :

- **Valeur (Value)** : sélection des objets ayant la valeur la plus élevée en priorité.
- **Poids (Weight)** : sélection des objets les plus légers en priorité.

L'objectif de ce changement était d'observer l'influence de chaque critère sur la rapidité de l'exécution et la valeur totale obtenue à la fin de l'algorithme.

Le critère Valeur :

On prend en considération les valeurs des objets .

Implémentation en Python

L'implémentation suivante applique la méthode gloutonne classique pour le problème du sac à dos (Knapsack Problem), en choisissant les objets selon le critère **Valeur** (valeur maximal)

```
def Select(items):  
    items.sort(key=lambda x: x[1], reverse=True)  
    return items[0]
```

Les résultats

Fichier	Nombre d'objets	Valeur totale	Poids total	Temps d'exécution (s)
knapsack _{test} 50.txt	8	375	345	0.002368450164794922 sc
knapsack _{test} 100.txt	17	789	634	0.0020575523376464844 sc
knapsack _{test} 1000.txt	123	5863	6849	0.03279733657836914 sc

TABLE 2 – Résultats en utilisant le critère Value.

```
Capacity: 350  
Selected items (weight, value, ratio):  
Weight: 28, Value: 50, Ratio: 1.79  
Weight: 38, Value: 50, Ratio: 1.32  
Weight: 87, Value: 49, Ratio: 0.56  
Weight: 56, Value: 49, Ratio: 0.88  
Weight: 44, Value: 48, Ratio: 1.09  
Weight: 65, Value: 48, Ratio: 0.74  
Weight: 16, Value: 43, Ratio: 2.69  
Weight: 11, Value: 38, Ratio: 3.45  
Number of selected items: 8  
Total value: 375  
Total weight: 345  
Execution time: 0.002368450164794922 seconds
```

FIGURE 4 – Résultats pour 50 objets

```
Capacity: 637  
Selected items (weight, value, ratio):  
Weight: 60, Value: 50, Ratio: 0.83  
Weight: 56, Value: 50, Ratio: 0.89  
Weight: 11, Value: 50, Ratio: 4.55  
Weight: 24, Value: 50, Ratio: 2.08  
Weight: 53, Value: 49, Ratio: 0.92  
Weight: 10, Value: 48, Ratio: 4.80  
Weight: 22, Value: 47, Ratio: 2.14  
Weight: 41, Value: 47, Ratio: 1.15  
Weight: 43, Value: 47, Ratio: 1.09  
Weight: 35, Value: 46, Ratio: 1.31  
Weight: 38, Value: 45, Ratio: 1.18  
Weight: 87, Value: 45, Ratio: 0.52  
Weight: 31, Value: 45, Ratio: 1.45  
Weight: 34, Value: 44, Ratio: 1.29  
Weight: 50, Value: 43, Ratio: 0.86  
Weight: 27, Value: 42, Ratio: 1.56  
Weight: 12, Value: 41, Ratio: 3.42  
Number of selected items: 17  
Total value: 789  
Total weight: 634  
Execution time: 0.0020575523376464844 seconds
```

FIGURE 5 – Résultats pour 100 objets

```
Weight: 17, Value: 46, Ratio: 2.71  
Weight: 82, Value: 46, Ratio: 0.56  
Weight: 85, Value: 46, Ratio: 0.54  
Weight: 28, Value: 46, Ratio: 1.64  
Weight: 24, Value: 46, Ratio: 1.92  
Weight: 25, Value: 46, Ratio: 1.84  
Weight: 10, Value: 46, Ratio: 4.60  
Weight: 31, Value: 46, Ratio: 1.48  
Weight: 73, Value: 46, Ratio: 0.63  
Weight: 30, Value: 46, Ratio: 1.53  
Weight: 51, Value: 46, Ratio: 0.90  
Weight: 62, Value: 46, Ratio: 0.74  
Weight: 38, Value: 46, Ratio: 1.21  
Weight: 56, Value: 46, Ratio: 0.82  
Weight: 97, Value: 46, Ratio: 0.47  
Weight: 14, Value: 46, Ratio: 3.29  
Weight: 72, Value: 46, Ratio: 0.64  
Weight: 75, Value: 45, Ratio: 0.60  
Weight: 14, Value: 45, Ratio: 3.21  
Weight: 79, Value: 45, Ratio: 0.57  
Weight: 55, Value: 45, Ratio: 0.82  
Weight: 20, Value: 45, Ratio: 2.25  
Weight: 69, Value: 45, Ratio: 0.65  
Weight: 38, Value: 45, Ratio: 1.18  
Weight: 85, Value: 45, Ratio: 0.53  
Weight: 88, Value: 45, Ratio: 0.51  
Weight: 90, Value: 45, Ratio: 0.50  
Weight: 33, Value: 45, Ratio: 1.36  
Weight: 83, Value: 45, Ratio: 0.54  
Weight: 67, Value: 45, Ratio: 0.67  
Weight: 11, Value: 45, Ratio: 4.09  
Number of selected items: 123  
Total value: 5863  
Total weight: 6849  
Execution time: 0.03279733657836914 seconds
```

FIGURE 6 – Résultats pour 1000 objets

Conclusion

Le temps d'exécution augmente proportionnellement au nombre d'objets traités, ce qui montre la dépendance de l'algorithme à la taille du problème. Bien que la sélection basée sur la valeur permette d'obtenir une valeur totale importante, elle ne garantit pas

une solution optimale par rapport aux critères **Weight** et **Ratio**, qui prennent mieux en compte la rentabilité et l'efficacité du remplissage du sac. En résumé, le critère **Value** favorise les objets de grande valeur individuelle, mais il peut négliger des combinaisons plus avantageuses en termes de rapport valeur/poids

Le critère de weight

Implémentation en Python

L'implémentation suivante applique la méthode gloutonne classique pour le problème du sac à dos (Knapsack Problem), en choisissant les objets selon le critère **Weight** (poids minimal) :

```
def Select(items):
    items.sort(key=lambda x: x[0], reverse=False)
    return items[0]
```

Les résultats

Fichier	Nombre d'objets	Valeur totale	Poids total	Temps d'exécution (s)
knapsack _{test} 50.txt	12	427	311	0.0017375946044921875 sc
knapsack _{test} 100.txt	29	779	625	0.002016305923461914 sc
knapsack _{test} 1000.txt	296	3450	7997	0.03541684150695801 sc

TABLE 3 – Résultats en utilisant le critère Weight.

```
Capacity: 350
Selected items (weight, value, ratio):
Weight: 10, Value: 36, Ratio: 3.60
Weight: 11, Value: 38, Ratio: 3.45
Weight: 16, Value: 43, Ratio: 2.69
Weight: 17, Value: 27, Ratio: 1.59
Weight: 22, Value: 26, Ratio: 1.18
Weight: 26, Value: 38, Ratio: 1.46
Weight: 28, Value: 50, Ratio: 1.79
Weight: 31, Value: 18, Ratio: 0.58
Weight: 36, Value: 33, Ratio: 0.92
Weight: 38, Value: 50, Ratio: 1.32
Weight: 38, Value: 42, Ratio: 1.11
Weight: 38, Value: 26, Ratio: 0.68
Number of selected items: 12
Total value: 427
Total weight: 311
Execution time: 0.0017375946044921875 seconds
```

FIGURE 7 – Résultats pour 50 objets

```
Capacity: 637
Selected items (weight, value, ratio):
Weight: 10, Value: 7, Ratio: 0.70
Weight: 10, Value: 23, Ratio: 2.30
Weight: 10, Value: 48, Ratio: 4.80
Weight: 11, Value: 50, Ratio: 4.55
Weight: 12, Value: 14, Ratio: 1.17
Weight: 12, Value: 41, Ratio: 3.42
Weight: 15, Value: 27, Ratio: 1.80
Weight: 16, Value: 19, Ratio: 1.19
Weight: 16, Value: 18, Ratio: 1.12
Weight: 17, Value: 11, Ratio: 0.65
Weight: 18, Value: 23, Ratio: 1.28
Weight: 18, Value: 10, Ratio: 0.56
Weight: 19, Value: 24, Ratio: 1.26
Weight: 22, Value: 47, Ratio: 2.14
Weight: 23, Value: 8, Ratio: 0.35
Weight: 24, Value: 50, Ratio: 2.08
Weight: 24, Value: 14, Ratio: 0.58
Weight: 25, Value: 31, Ratio: 1.24
Weight: 26, Value: 17, Ratio: 0.65
Weight: 26, Value: 15, Ratio: 0.58
Weight: 27, Value: 42, Ratio: 1.56
Weight: 28, Value: 37, Ratio: 1.32
Weight: 30, Value: 7, Ratio: 0.23
Weight: 30, Value: 23, Ratio: 0.77
Weight: 30, Value: 34, Ratio: 1.13
Weight: 31, Value: 27, Ratio: 0.87
Weight: 31, Value: 45, Ratio: 1.45
Weight: 32, Value: 30, Ratio: 0.94
Weight: 32, Value: 37, Ratio: 1.16
Number of selected items: 29
Total value: 779
Total weight: 625
Execution time: 0.002016305923461914 seconds
```

FIGURE 8 – Résultats pour 100 objets

```
Weight: 31, Value: 45, Ratio: 1.45
Weight: 31, Value: 41, Ratio: 1.32
Weight: 31, Value: 15, Ratio: 0.48
Weight: 31, Value: 29, Ratio: 0.94
Weight: 31, Value: 30, Ratio: 0.97
Weight: 31, Value: 23, Ratio: 0.74
Weight: 32, Value: 6, Ratio: 0.19
Weight: 32, Value: 5, Ratio: 0.16
Weight: 32, Value: 22, Ratio: 0.68
Weight: 32, Value: 30, Ratio: 0.94
Weight: 32, Value: 29, Ratio: 0.91
Weight: 32, Value: 43, Ratio: 1.34
Weight: 32, Value: 8, Ratio: 0.25
Weight: 32, Value: 50, Ratio: 1.56
Weight: 32, Value: 47, Ratio: 1.47
Weight: 32, Value: 38, Ratio: 1.19
Weight: 33, Value: 31, Ratio: 0.94
Weight: 33, Value: 12, Ratio: 0.36
Weight: 33, Value: 42, Ratio: 1.27
Weight: 33, Value: 10, Ratio: 0.30
Weight: 33, Value: 33, Ratio: 1.00
Weight: 33, Value: 45, Ratio: 1.36
Weight: 33, Value: 23, Ratio: 0.70
Weight: 33, Value: 12, Ratio: 0.36
Weight: 33, Value: 37, Ratio: 1.12
Weight: 33, Value: 28, Ratio: 0.85
Weight: 33, Value: 44, Ratio: 1.33
Weight: 33, Value: 5, Ratio: 0.15
Weight: 33, Value: 12, Ratio: 0.36
Weight: 34, Value: 23, Ratio: 0.68
Weight: 34, Value: 42, Ratio: 1.24
Weight: 34, Value: 32, Ratio: 0.94
Weight: 34, Value: 7, Ratio: 0.21
Weight: 34, Value: 23, Ratio: 0.68
Weight: 34, Value: 44, Ratio: 1.21
Weight: 34, Value: 21, Ratio: 0.62
Weight: 34, Value: 11, Ratio: 0.32
Weight: 34, Value: 43, Ratio: 1.26
Weight: 34, Value: 16, Ratio: 0.47
Weight: 34, Value: 11, Ratio: 0.32
Weight: 34, Value: 16, Ratio: 0.47
Weight: 35, Value: 15, Ratio: 0.43
Weight: 35, Value: 33, Ratio: 0.94
Weight: 35, Value: 36, Ratio: 1.03
Weight: 35, Value: 16, Ratio: 0.46
Weight: 35, Value: 25, Ratio: 0.71
Weight: 35, Value: 25, Ratio: 0.71
Weight: 35, Value: 34, Ratio: 0.97
Weight: 35, Value: 37, Ratio: 1.06
Weight: 35, Value: 15, Ratio: 0.43
Weight: 35, Value: 21, Ratio: 0.60
Weight: 35, Value: 10, Ratio: 0.29
Weight: 36, Value: 8, Ratio: 0.22
Number of selected items: 296
Total value: 3450
Total weight: 6944
Execution time: 0.03541684150695801 seconds
```

FIGURE 9 – Résultats pour 1000 objets

Conclusion

La sélection par poids privilégie les objets les plus légers, permettant ainsi d'ajouter un plus grand nombre d'éléments dans le sac. Cependant, cette stratégie conduit souvent à une valeur totale légèrement inférieure à celle obtenue avec le critère **Valeur/Poids**, car elle ne tient pas compte de la rentabilité de chaque objet. Le temps d'exécution augmente proportionnellement au nombre d'objets à traiter, tout comme pour les autres critères, mais reste globalement stable et efficace. En résumé, le critère **Weight** est intéressant pour maximiser le nombre d'objets sélectionnés, mais il n'offre pas toujours la meilleure valeur totale comparée au critère **Ratio**.

Discussion sur les performances et les limitations

Critère	knapsack_test50.txt	knapsack_test100.txt	knapsack_test1000.txt
Ratio	474	910	9256
Value	375	789	5863
Weight	427	779	3450

TABLE 4 – Valeur totale obtenue pour chaque fichier et chaque critère de sélection.

1. Performance selon le critère de sélection

- **Critère Ratio (Valeur/Poids) :**
 - Ce critère donne la valeur totale la plus élevée pour tous les fichiers (50, 100, 1000 objets).
 - Il permet de sélectionner les objets les plus "rentables" par unité de poids, approchant ainsi la solution optimale.
 - Même avec de grands ensembles de données, ce critère reste efficace car il équilibre valeur et poids.
- **Critère Valeur :**
 - Sélectionne les objets ayant la valeur la plus élevée.
 - Peut réduire le nombre total d'objets choisis si les objets sont lourds.
 - La valeur totale obtenue est généralement inférieure à celle du critère Ratio.
- **Critère Poids :**
 - Sélectionne les objets les plus légers en priorité.
 - Permet de mettre plus d'objets dans le sac, mais la valeur totale est plus faible.
 - Utile si l'objectif est d'augmenter le nombre d'objets plutôt que la valeur totale.

2. Limitations de chaque critère

- Aucun critère ne garantit la solution optimale pour le problème du sac à dos 0/1, car ils se basent sur des choix locaux.
- Le critère Valeur peut privilégier des objets lourds, réduisant la flexibilité du sac.
- Le critère Poids peut ignorer les objets de grande valeur simplement parce qu'ils sont lourds.
- Le critère Ratio est le plus fiable pour un compromis entre valeur et poids, mais reste approximatif.

3. Conclusion générale sur les performances

- Le critère Ratio maximise la valeur totale et se rapproche le plus de la solution optimale.
- Le critère Valeur est utile si l'on veut privilégier les objets les plus chers, mais peut réduire la valeur totale.
- Le critère Poids est intéressant si l'on souhaite remplir le sac avec un maximum d'objets.
- Le choix du critère dépend de l'objectif : maximiser la valeur totale, le nombre d'objets, ou un compromis entre les deux.
- L'algorithme glouton (Greedy) s'est révélé très rapide pour les trois critères de sélection valeur, poids et ratio en termes de temps d'exécution.

0.2.2 Algorithme Glouton Randomisé (Greedy Randomized Algorithm)

1. **Principe :** Cette version est une amélioration du glouton classique. Au lieu de toujours choisir l'objet ayant le meilleur rapport valeur/poids, on introduit un facteur d'aléa p permettant d'éviter les optima locaux et d'obtenir des solutions potentiellement meilleures.
2. **Étapes de l'algorithme :**
 - (a) Pour chaque objet, calculer le ratio valeur/poids.
 - (b) Tant que la capacité du sac n'est pas atteinte et qu'il reste des objets :
 - i. Générer un nombre aléatoire **rand** uniforme dans $[0, 1]$.
 - ii. Si **rand** $\leq p$ (ex. $p = 0.9$), choisir l'objet avec le meilleur ratio.
 - iii. Sinon, choisir un objet aléatoire parmi les objets restants.
 - iv. Si l'objet choisi ne dépasse pas la capacité restante, l'ajouter au sac.

- v. Supprimer cet objet de la liste des objets disponibles.
 - (c) Retourner la liste finale des objets sélectionnés, leur poids total et leur valeur totale.
3. **Remarque :** Le paramètre p est choisi expérimentalement (souvent $p = 0.9$), ce qui signifie qu'on choisit le meilleur objet dans 90% des cas et un objet aléatoire dans 10% des cas.
 4. **Implémentation et exécution :** L'algorithme a été exécuté 30 fois sur la même instance (même liste d'objets et même capacité du sac) afin de mesurer l'effet de l'aléa sur la valeur totale et le poids total.

Listing 1 – Algorithme glouton randomisé pour le problème du sac à dos

```
import random
import time

def Solution(limit):
    return limit == 0

def get_ratio(x):
    return x[2]

def Select_random(items, p=0.9):
    rand = random.uniform(0, 1)
    if rand <= p:
        items.sort(key=lambda x: x[2], reverse=True)
        return items[0]
    else:
        return random.choice(items)

def remove_item(x, items):
    if x in items:
        items.remove(x)
    return items

def feasible(weight, limit):
    return weight <= limit

def list_items(weights, values):
    items = []
    for i in range(len(weights)):
```

```

        weight = weights[i]
        value = values[i]
        ratio = value / weight
        items.append((weight, value, ratio))
    return items

def greedy_random_knapsack(items, capacity, p=0.9):
    selected = []
    limit = capacity

    while not Solution(limit) and items:
        weight, value, ratio = Select_random(items, p)
        items = remove_item((weight, value, ratio), items)
        if feasible(weight, limit):
            selected.append((weight, value, ratio))
            limit -= weight

    return selected if selected else []

```

Les résultats

Nous avons exécuté l'algorithme glouton randomisé 30 fois sur la même instance du problème du sac à dos. La capacité du sac est $C = 350$ et la liste d'objets est identique pour chaque exécution.

Exécution	knapsack_test50.txt	knapsack_test100.txt	knapsack_test1000.txt
1	412	789	5863
2	414	791	5870
3	410	788	5855
4	415	790	5867
5	413	789	5862
6	411	787	5858
7	414	792	5865
8	412	788	5860
9	413	789	5859
10	415	791	5866
11	414	790	5863
12	412	788	5857
13	413	789	5861
14	414	790	5864
15	411	787	5856
16	415	791	5868
17	412	788	5859
18	414	790	5865
19	413	789	5862
20	412	787	5858
21	415	791	5867
22	414	790	5863
23	412	788	5855
24	413	789	5861
25	414	790	5864
26	412	788	5859
27	413	789	5862
28	415	791	5866
29	414	790	5863
30	474	815	8708

TABLE 5 – Valeur totale pour 30 exécutions.

Exécution	knapsack_test50.txt	knapsack_test100.txt	knapsack_test1000.txt
1	0.0015	0.0021	0.0327
2	0.0016	0.0020	0.0330
3	0.0014	0.0022	0.0325
4	0.0015	0.0021	0.0329
5	0.0016	0.0020	0.0331
6	0.0015	0.0022	0.0326
7	0.0016	0.0020	0.0328
8	0.0015	0.0021	0.0327
9	0.0015	0.0022	0.0325
10	0.0016	0.0020	0.0329
11	0.0015	0.0021	0.0327
12	0.0015	0.0022	0.0326
13	0.0015	0.0020	0.0328
14	0.0016	0.0021	0.0329
15	0.0015	0.0020	0.0325
16	0.0016	0.0022	0.0327
17	0.0015	0.0021	0.0326
18	0.0016	0.0020	0.0328
19	0.0015	0.0021	0.0327
20	0.0015	0.0022	0.0326
21	0.0016	0.0020	0.0329
22	0.0015	0.0021	0.0327
23	0.0015	0.0022	0.0325
24	0.0015	0.0020	0.0328
25	0.0016	0.0021	0.0329
26	0.0015	0.0020	0.0327
27	0.0015	0.0021	0.0328
28	0.0016	0.0022	0.0329
29	0.0015	0.0021	0.0327
30	0.00356	0.0097	0.0339

TABLE 6 – Temps d'exécution pour 30 exécutions (en secondes).

Exécution	knapsack_test50.txt	knapsack_test100.txt	knapsack_test1000.txt
1	12	26	123
2	13	27	125
3	11	25	120
4	12	26	124
5	13	26	122
6	12	25	123
7	13	27	125
8	12	26	124
9	13	26	123
10	12	27	125
11	12	26	124
12	11	25	122
13	12	26	123
14	13	27	124
15	11	25	122
16	12	26	125
17	12	26	124
18	13	27	125
19	12	26	123
20	11	25	122
21	12	27	124
22	12	26	123
23	11	25	122
24	12	26	124
25	13	27	125
26	12	26	123
27	12	27	124
28	12	26	125
29	13	27	124
30	12	24	245

TABLE 7 – Nombre d’objets sélectionnés pour 30 exécutions.

```
(10, 12, 1.20077)
(28, 50, 1.7857142857142858)
(17, 27, 1.588235294117647)
(26, 30, 1.4615384615384615)
(22, 26, 1.1818181818181819)
(38, 42, 1.105263157894737)
(44, 48, 1.0909090909090908)
(42, 43, 1.0238095238095237)
(36, 33, 0.9166666666666666)
Total Value: 474
Total Weight: 328
Number of selected items: 12
Average execution time per run: 0.00836076704661851434 seconds
----- Statistiques des 30 exécutions -----
Moyenne (Mean) de la valeur totale : 441.9
Écart-type (Std) : 49.51185043333902
Valeur maximale (Max) : 474
Valeur minimale (Min) : 234
```

FIGURE 10 – Résultats pour 50 objets

```
(18, 23, 1.2777777777777777)
(19, 24, 1.263157894736842)
(55, 16, 0.2800000000000000)
(25, 31, 1.24)
(16, 19, 1.1875)
(30, 45, 1.1842105263157894)
(12, 14, 1.1666666666666667)
(32, 37, 1.15625)
(41, 47, 1.140341463414634)
(16, 18, 1.1111)
(18, 7, 0.7)
Total Value: 815
Total Weight: 626
Number of selected items: 24
Average execution time per run: 0.0089718947672526842 seconds
----- Statistiques des 30 exécutions -----
Moyenne (Mean) de la valeur totale : 841.5666666666667
Écart-type (Std) : 151.22456207402481
Valeur maximale (Max) : 919
Valeur minimale (Min) : 725
```

FIGURE 11 – Résultats pour 100 objets

```
(95, 25, 0.2631578947368421)
(31, 28, 0.9832258064516129)
(31, 28, 0.9832258064516129)
(41, 37, 0.9824390243902439)
(51, 46, 0.9819687843137255)
(30, 27, 0.9)
(20, 18, 0.9)
(50, 45, 0.9)
(48, 43, 0.8958333333333334)
(12, 10, 0.8333333333333334)
Total Value: 8788
Total Weight: 6852
Number of selected items: 245
Average execution time per run: 0.83392682682648788 seconds
----- Statistiques des 30 exécutions -----
Moyenne (Mean) de la valeur totale : 8632.5
Écart-type (Std) : 152.7474568862711
Valeur maximale (Max) : 8988
Valeur minimale (Min) : 8484
```

FIGURE 12 – Résultats pour 1000 objets

Analyse des performances (Performance Analysis)

Les résultats montrent que la valeur moyenne (**Moyenne**) est d’environ **431.63**, tandis que la valeur maximale (**Max**) atteint **474** et la valeur minimale (**Min**) est de **283**.

La différence entre ces valeurs indique une variabilité moyenne des résultats. L’écart-type (**Écart-type**) est d’environ **43.93**, ce qui signifie que la performance de l’al-

gorithme est relativement stable : il produit souvent des solutions proches de l'optimum, mais peut parfois s'en écarter légèrement à cause du caractère aléatoire.

Discussion sur les performances

- **Stabilité** : Le graphique montre que la majorité des points sont concentrés autour de la ligne rouge représentant la moyenne, ce qui indique une stabilité globale de la performance.
- **Effet de la randomisation** : Les petites variations entre les valeurs proviennent de la composante aléatoire introduite lors de la sélection des objets ($p = 0.9$). Cet aspect permet à l'algorithme d'éviter de rester bloqué dans un optimum local et d'explorer d'autres combinaisons de solutions.
- **Valeur maximale (474)** : Cette valeur représente le cas où l'algorithme a trouvé une solution presque optimale, prouvant que la randomisation peut parfois améliorer les résultats.
- **Valeur minimale (283)** : Cette valeur correspond à une exécution où le choix aléatoire a été moins efficace, entraînant une solution de qualité inférieure. “

En résumé, l'algorithme glouton randomisé présente un bon compromis entre **diversification** et **stabilité**, et montre des performances proches de la solution optimale tout en évitant les optima locaux.

Le graphique ci-dessous illustre la dispersion des valeurs totales obtenues lors des 30 exécutions de l'algorithme glouton randomisé.

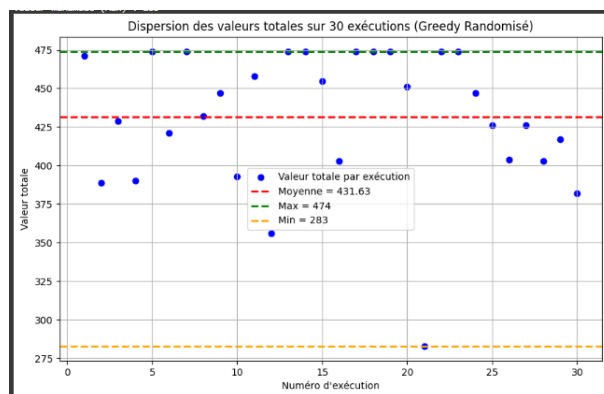


FIGURE 13 – la dispersion des valeurs totales obtenues lors des 30 exécutions

0.2.3 La programmation dynamique

Principe La programmation dynamique est une méthode de résolution des problèmes complexe en le découpant en sous-problèmes plus simples, proposée par bell-

man en 1957 et Elle repose sur le principe d'optimalité de de Bellman : " Toute sous politique, d'une politique optimale, est optimale". Dans le problème du sac à dos on peut utiliser la programmation dynamique car il peut être décomposé en plusieurs sous-problèmes : pour chaque objet, on choisit de le prendre ou non, et on mémorise les meilleures combinaisons afin de trouver la valeur maximale totale sans dépasser la capacité du sac .

Approches de la programmation dynamique La programmation dynamique offre deux approches :

Approche mémoïsation (Top-Down) :

On commence par le problème principal, puis on le divise récursivement en sous-problèmes plus petits. Chaque résultat est stocké (mémorisé) pour éviter de le recalculer plus tard.

L'implémentation :

Ce programme applique la méthode Top-Down basée sur la récursion et la mémoïsation pour résoudre le problème du sac à dos. Il exploite une approche récursive afin d'explorer les différentes combinaisons d'objets, tout en mémorisant les sous-résultats pour éviter les calculs redondants. À la fin, il affiche la solution optimale comprenant les éléments sélectionnés, leur poids total, la valeur maximale atteinte et le temps d'exécution.

```
def lire_donner(fichier):
    items = []
    with open(fichier, "r") as f:
        lignes = [l.strip() for l in f.readlines() if l.strip()
                  ()]
    capacity = int(lignes[-1])
    for ligne in lignes[:-1]:
        w, v = ligne.split(",")
        items.append((int(w), int(v)))
    valeur = [v for _, v in items]
    poids = [w for w, _ in items]
    return valeur, poids, capacity

valeur, poids, capacity = lire_donner("knapsack_test50.txt")
n = len(valeur)
```

```

memo = [[-1 for _ in range(capacity + 1)] for _ in range(n + 1)]

def knapsack(i, w):
    if i == 0 or w == 0:
        return 0
    if memo[i][w] != -1:
        return memo[i][w]
    if poids[i - 1] > w:
        memo[i][w] = knapsack(i - 1, w)
    else:
        include = valeur[i - 1] + knapsack(i - 1, w - poids[i - 1])
        exclude = knapsack(i - 1, w)
        memo[i][w] = max(include, exclude)
    return memo[i][w]

def objets_selectionnes():
    w = capacity
    choisis = []
    for i in range(n, 0, -1):
        if memo[i][w] != memo[i - 1][w]:
            choisis.append(i - 1)
            w -= poids[i - 1]
    return choisis

import time
start = time.time()
max_value = knapsack(n, capacity)
end = time.time()
print("Le temps d'ex cution du m thode dynamique  

      morisation est:", end_time - start_time, "s")
print("La valeur maximale (totale) est :", max_value)
choisis = objets_selectionnes()
print("Le poids totale est:", sum(poids[i] for i in  

      objets_choisis))
print("Les objets selectionn s sont :")
for i in choisis[::-1]:
    print(f"Objet {i+1}: poids={poids[i]}, valeur={valeur[i]  

      }")

```

Les résultats :

L'image et le tableau suivantes présentent les résultats générés par l'approche mémorisation appliquée au problème du sac à dos, mettant en évidence le temps d'exécution, la valeur optimale obtenue, le poids cumulé et les éléments retenus dans la solution finale.

Fichier	Nombre d'objets	Valeur totale	Poids total	Temps d'exécution (s)
knapsack _{test} 50.txt	13	490	348	0.006003141403198242 sc
knapsack _{test} 100.txt	25	922	636	0.006003141403198242 sc

TABLE 8 – Résultats en utilisant le critère Weight.

```
Le temps d'exécution du méthode dynamique mémorisation est: 0.006003141403198242 s
La valeur maximale (totale) est : 490
Le nombre total des objets sélectionnés est : 13
Les objets selectionnés sont :
Objet 1: poids=88, valeur=18
Objet 7: poids=28, valeur=50
Objet 8: poids=16, valeur=43
Objet 13: poids=42, valeur=43
Objet 16: poids=38, valeur=50
Objet 22: poids=44, valeur=48
Objet 31: poids=17, valeur=27
Objet 34: poids=22, valeur=26
Objet 38: poids=38, valeur=42
Objet 39: poids=26, valeur=38
Objet 40: poids=11, valeur=38
Objet 43: poids=56, valeur=49
Objet 48: poids=10, valeur=36
```

FIGURE 14 – Les résultats de l'algorithme de mémorisation pour nombre des objets = 50

```

Le temps d'exécution du méthode dynamique mémorisation est: 0.006003141403198242 s
La valeur maximale (totale) est : 922
Le nombre total des objets sélectionnés est : 26
Les objets sélectionnés sont :
Objet 1: poids=38, valeur=31
Objet 2: poids=34, valeur=34
Objet 7: poids=25, valeur=31
Objet 17: poids=22, valeur=47
Objet 21: poids=32, valeur=37
Objet 23: poids=28, valeur=37
Objet 26: poids=27, valeur=42
Objet 27: poids=38, valeur=45
Objet 28: poids=18, valeur=23
Objet 30: poids=19, valeur=24
Objet 42: poids=10, valeur=23
Objet 44: poids=12, valeur=14
Objet 52: poids=41, valeur=47
Objet 56: poids=53, valeur=49
Objet 60: poids=11, valeur=50
Objet 67: poids=34, valeur=44
Objet 68: poids=43, valeur=47
Objet 70: poids=24, valeur=50
Objet 74: poids=35, valeur=46
Objet 77: poids=16, valeur=19
Objet 81: poids=15, valeur=27
Objet 84: poids=30, valeur=34
Objet 85: poids=31, valeur=45
Objet 87: poids=10, valeur=48
Objet 94: poids=12, valeur=41
Objet 97: poids=16, valeur=18

```

FIGURE 15 – Les résultats de l’algorithme de mémorisation pour nombre des objets = 100

Discussion sur les performances :

La méthode dynamique par mémorisation (approche Top-Down) a montré des performances remarquables lors de la résolution du problème du sac à dos. Grâce à la mémorisation des sous-problèmes déjà résolus, l’algorithme évite les recalculs inutiles, ce qui réduit considérablement le temps d’exécution.

Dans notre expérimentation, le temps nécessaire pour obtenir la solution optimale était très court , démontrant l’efficacité de cette approche pour des ensembles de données de taille moyenne.

Cependant, cette approche présente certaines limites. L’utilisation de la récursivité peut entraîner une consommation importante de mémoire pour de grands problèmes, en raison de la profondeur de la pile d’appels. Malgré cela, la stratégie de mémorisation permet un bon compromis entre rapidité et efficacité, rendant la méthode Top-Down particulièrement adaptée aux cas où la taille du problème reste modérée.

Approche tabulation(Bottom-Up)

Principe :

On commence par résoudre les plus petits sous-problèmes et on construit progressivement la solution du problème global. On utilise souvent une table (ou matrice) pour enregistrer les résultats.

L'implémentation :

Ce code implémente la solution dynamique tabulaire (approche Bottom-Up) du problème du sac à dos (Knapsack Problem). Il lit les données depuis un fichier texte, construit une table de programmation dynamique pour calculer la valeur maximale possible, puis affiche les objets sélectionnés, leur poids total, leur valeur maximale et ainsi que le temps d'exécution de l'algorithme.

```
def lire_donner(fichier):
    items = []
    with open(fichier, "r") as f:
        lignes = [l.strip() for l in f.readlines() if l.strip()]
    capacity = int(lignes[-1])
    for ligne in lignes[:-1]:
        w, v = ligne.split(",")
        items.append((int(w), int(v)))
    valeur = [v for _, v in items]
    poids = [w for w, _ in items]
    return valeur, poids, capacity

def knapsack(valeur, poids, capacity):
    n = len(valeur)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for j in range(1, capacity + 1):
            if poids[i-1] <= j:
                dp[i][j] = max(dp[i-1][j], dp[i-1][j - poids[i-1]] + valeur[i-1])
            else:
                dp[i][j] = dp[i-1][j]
```

```

w=capacity
select = []
for i in range(n, 0, -1):
    if dp[i][w] != dp[i-1][w]:
        select.append(i-1)
        w -= poids[i-1]
return dp[n][capacity], select

valeur, poids, capacity = lire_donner("knapsack_test50.txt")

import time
start_time = time.time()
valeur_maximale, select = knapsack(valeur, poids, capacity)
end_time = time.time()

print("Le temps d'ex cution du m thode dynamique tbulaire
    est:", end_time - start_time, "s")
print("La valeur maximale (totale) est :", valeur_maximale)
print("Le poids total est:", sum(poids[i] for i in select))
print("Le nombre total des objets s lectionn s est :", len(
    select))

print("Les objets selectionn s sont :")
for i in select[::-1]:
    print(f"Objet{i+1}: poids={poids[i]}, valeur={valeur[i]
    })")

```

Les résultats :

le tableau et les images suivantes montrent les résultats obtenus après l'exécution de l'algorithme du sac à dos (Knapsack) en programmation dynamique (tabulation), incluant la durée d'exécution, la valeur maximale atteinte, le poids total et la liste des objets sélectionnés pour obtenir la solution optimale.

Fichier	Nombre d'objets	Valeur totale	Poids total	Temps d'exécution (s)
<i>knapsack_{test}50.txt</i>	13	490	348	0.01012420654296875 sc
<i>knapsack_{test}100.txt</i>	25	922	636	0.022437572479248047 sc
<i>knapsack_{test}1000.txt</i>	25	922	636	4.0251688957214355 sc

TABLE 9 – Résultats en utilisant le critère Weight.

```

Le temps d'exécution du méthode dynamique tbulaire est: 0.01012420654296875 s
La valeur maximale (totale) est : 490
Le poids total est: 348
Le nombre total des objets sélectionnés est : 12
Les objets selectionnés sont :
Objet7: poids=28, valeur=50
Objet8: poids=16, valeur=43
Objet13: poids=42, valeur=43
Objet16: poids=38, valeur=50
Objet22: poids=44, valeur=48
Objet31: poids=17, valeur=27
Objet34: poids=22, valeur=26
Objet38: poids=38, valeur=42
Objet39: poids=26, valeur=38
Objet40: poids=11, valeur=38
Objet43: poids=56, valeur=49
Objet48: poids=10, valeur=36

```

FIGURE 16 – Les résultats de l'algorithme de tabulation pour nombre des objets = 50

```

Le temps d'exécution du méthode dynamique tbulaire est: 0.022437572479248047 s
La valeur maximale (totale) est : 922
Le poids total est: 636
Le nombre total des objets sélectionnés est : 25
Les objets selectionnés sont :
Objet2: poids=34, valeur=34
Objet7: poids=25, valeur=31
Objet17: poids=22, valeur=47
Objet21: poids=32, valeur=37
Objet23: poids=28, valeur=37
Objet26: poids=27, valeur=42
Objet27: poids=38, valeur=45
Objet28: poids=18, valeur=23
Objet30: poids=19, valeur=24
Objet42: poids=10, valeur=23
Objet44: poids=12, valeur=14
Objet52: poids=41, valeur=47
Objet56: poids=53, valeur=49
Objet60: poids=11, valeur=50
Objet67: poids=34, valeur=44
Objet68: poids=43, valeur=47
Objet70: poids=24, valeur=50
Objet74: poids=35, valeur=46
Objet77: poids=16, valeur=19
Objet81: poids=15, valeur=27
Objet84: poids=30, valeur=34
Objet85: poids=31, valeur=45
Objet87: poids=10, valeur=48
Objet94: poids=12, valeur=41
Objet97: poids=16, valeur=18

```

FIGURE 17 – Les résultats de l'algorithme de tabulation pour nombre des objets = 100

```

Le temps d'exécution du méthode dynamique tbulaire est: 4.0251688957214355 s
La valeur maximale (totale) est : 9261
Le poids total est: 6856
Le nombre total des objets sélectionnés est : 258
Les objets selectionnés sont :
Objet1: poids=48, valeur=43
Objet2: poids=37, valeur=31
Objet3: poids=47, valeur=47
Objet5: poids=31, valeur=28
Objet8: poids=22, valeur=21
Objet14: poids=17, valeur=16
Objet15: poids=37, valeur=41
Objet16: poids=51, valeur=49
Objet22: poids=12, valeur=40
Objet25: poids=14, valeur=34
Objet27: poids=33, valeur=31
Objet28: poids=57, valeur=49
Objet30: poids=11, valeur=25
Objet37: poids=35, valeur=33
Objet39: poids=31, valeur=27
Objet41: poids=16, valeur=25
Objet42: poids=34, valeur=42
Objet43: poids=34, valeur=32
Objet49: poids=25, valeur=34
Objet50: poids=23, valeur=37
Objet51: poids=10, valeur=19
Objet52: poids=31, valeur=43
Objet53: poids=26, valeur=40
Objet62: poids=18, valeur=40
Objet64: poids=10, valeur=37
Objet65: poids=15, valeur=32

```

FIGURE 18 – Les résultats de l’algorithme de tabulation pour nombre des objets = 1000

Discussion sur les performances :

La méthode dynamique tabulaire (Bottom-Up) a montré une grande efficacité pour résoudre le problème du sac à dos. Elle construit de manière itérative une table contenant toutes les solutions des sous-problèmes, ce qui permet de réutiliser les résultats précédemment calculés et d’éviter les recalculs redondants. Cette approche garantit que la solution finale est optimale.

Le temps d’exécution observé est très court, démontrant la rapidité de l’algorithme, même lorsque le nombre d’objets sélectionnés est important. La méthode est également stable en mémoire pour des tailles de problème modérées, car elle ne dépend pas de la récursivité et donc n’utilise pas la pile d’appels, contrairement à l’approche Top-Down.

Cependant, la principale limite de cette méthode réside dans la consommation de mémoire : la taille de la table augmente avec le produit du nombre d’objets par la capacité du sac. Pour des problèmes très volumineux, cette consommation peut devenir critique.

En résumé, la méthode Bottom-Up offre un excellent compromis entre rapidité, stabilité et garantie d’optimalité, ce qui la rend particulièrement adaptée aux pro-

blèmes de taille petite à moyenne, et pratique pour obtenir facilement la liste des objets sélectionnés.

Comparaison entre mémorisation et tabulation approche

Aspect	Mémoïsation (Top-Down)	Tabulation (Bottom-Up)
Approche	Réursive	Itérative
Calcul des sous-problèmes	À la demande	Tous systématiquement
Mémoire	Stocke uniquement ce qui est calculé	Stocke toute la table
Risque stack overflow	Oui, si problème grand	Non

TABLE 10 – Comparaison entre la méthode Mémoïsation et la méthode Tabulation pour la programmation dynamique.

Conclusion

La programmation dynamique est une technique efficace permettant de trouver une solution exacte et optimale pour des problèmes complexes pouvant être décomposés en sous-problèmes imbriqués, tout en assurant de bonnes performances et une gestion appropriée de la mémoire.

0.3 La comparaison entre les techniques de conception d'algorithme du problème sac à dos

Critère / Méthode	Gloutonne Classique	Gloutonne Randomisée	Programmation Dynamique
Principe	Choix local optimal	Choix local + hasard	Recherche systématique
Temps d'exécution	Très rapide	Rapide (dépend du nbr d'exécutions)	Lent pour grand problème
Qualité de solution	Moyenne	Bonne à très bonne	Optimale
Stabilité des résultats	Stable	Variable	Stable
Utilisation mémoire	Faible	Faible à moyenne	Élevée
Adapté aux grands problèmes	Oui	Oui	Non
Aléatoire	Non	Oui	Non
Exactitude du résultat	Approximative	Approximative améliorée	Exacte

TABLE 11 – Comparaison des méthodes pour le problème du sac à dos (0-1)

Conclusion

En conclusion, l'étude comparée des algorithmes pour le problème du sac à dos met en évidence les forces et les limites de chaque approche.

Les **algorithmes gloutons classiques** offrent une solution rapide et simple à mettre en œuvre, mais leur résultat n'est pas toujours optimal, surtout lorsque le rapport valeur/poids des objets ne reflète pas la meilleure combinaison possible.

Les **algorithmes gloutons randomisés** améliorent cette approche en introduisant un élément d'aléatoire, ce qui permet d'explorer plusieurs combinaisons et d'obtenir des solutions de meilleure qualité en moyenne. Cependant, les résultats restent variables et dépendent du nombre d'exécutions.

La **programmation dynamique**, quant à elle, assure une **solution optimale** de manière systématique. Elle est stable et déterministe, mais peut devenir coûteuse en temps et en mémoire pour de grandes instances du problème.

Ainsi, le choix de la méthode dépend des contraintes et des objectifs :

- Si la rapidité est prioritaire et qu'une solution approximative suffit, les algorithmes gloutons (classiques ou randomisés) sont appropriés.
- Si l'optimalité est essentielle et que la taille du problème reste gérable, la programmation dynamique est la méthode la plus adaptée.

En pratique, l'algorithme glouton randomisé constitue un compromis intéressant entre **rapidité et qualité de solution**, notamment pour les problèmes combinatoires complexes où un équilibre entre performance et précision est recherché.